

3. Project Design Phase

Project: PocketSmart AI - Smart Budget & Recommendation Assistant

3.1 Introduction

The Project Design Phase converts the requirements of PocketSmart AI into a structured technical and user-interface design. The project is designed as an intelligent, modular platform that provides personalized and budget-aware recommendations for Home Interior Planning, Party Planning, and Jewelry selection. The design connects the frontend, FastAPI backend, Gemini AI layer, user authentication, and recommendation workflow.

3.2 Overall System Design

The architecture described in the project documentation uses a frontend interface, a FastAPI backend, and a Gemini AI recommendation layer. The frontend collects budgets, preferences, contextual details, and optional images. The backend manages requests and routes, while the AI layer processes the information and generates structured recommendations.

3.3 Main Design Components

The major components are the Frontend UI, FastAPI Backend, Gemini AI Layer, planner modules, authentication services, session handling, and external product or service data sources. The frontend is implemented using HTML, CSS, JavaScript, and Jinja2 templates, while the backend is organized into modular routes, services, and models.

3.4 Design Goal

The main design goal is to create a clean and maintainable application in which each planner can collect domain-specific inputs and communicate with the common AI recommendation service. This modular approach keeps Home, Party, and Jewelry functions organized within the same application.

3.5 User-Centered Design

The user interface is designed to be lightweight, responsive, and intuitive. Users should be able to select a planner, enter the required details, submit the request, and view recommendations in a structured form. The documentation specifies card-like result displays for product suggestions, item details, and prices.

Page 2 - System Architecture Design

3.6 Frontend Layer

The Frontend UI is the first interaction layer. It provides pages and forms for collecting user information. The design includes a Home Page, authentication pages, planner pages, recommendation pages, a dashboard, testimonials, and recommendation history. Jinja2 templates are used to connect page content with backend data.

3.7 FastAPI Backend Layer

The FastAPI backend acts as the bridge between the frontend and the Gemini-based recommendation engine. Its design includes route handling, CORS configuration, session handling, startup processing, and structured input and output models. This layer keeps the application logic modular and easier to maintain.

3.8 Gemini AI Layer

The Gemini AI layer is responsible for interpreting the user's budget and context and generating personalized recommendations. The design also supports multimodal processing for the Jewelry Planner, where text and an optional outfit image can be supplied.

3.9 External Data Sources

The design allows recommendations to be associated with external product and service platforms such as Amazon, Flipkart, IKEA, Swiggy, Zomato, and OYO. The workflow also mentions mock API calls or simulated sourcing during development when direct data access is not available.

3.10 Modular Structure

The backend design separates responsibilities into routes, services, and models. Routes handle endpoints, services contain recommendation and AI-related logic, and models organize input and output schemas. The documentation also identifies `gemini_utils.py` for prompt orchestration, budget formatting, domain segmentation, and image analysis.

3.11 Communication Flow

The designed flow is: user enters planner information -> frontend sends the request -> FastAPI route receives the request -> planner-specific logic prepares the information -> Gemini AI processes the prompt -> recommendations are returned -> frontend displays the results. Authentication and session information can be included where required.

Page 3 - Planner Module Design

3.12 Home Interior Planner Design

The Home Interior Planner is designed around room and item requirements. The form can collect a budget, room type, and quantities for selected items. Example room types are Living Room, Kitchen, and Bedroom. Example items include lights, ceiling fans, and dining tables.

3.13 Home Recommendation Route

The /generate-home route is designed to receive home-related preferences and budget information. The recommendation logic then generates suitable home interior suggestions, considering functionality, style, and price.

3.14 Party Planner Design

The Party Planner is designed to collect total budget, guest count, event type, and venue details. The system organizes event requirements into areas such as catering, decoration, and entertainment, with accommodation services considered where applicable.

3.15 Party Recommendation Route

The /generate-party route is designed to process party details and guest count and return recommendations for food, venues, decorations, and related planning requirements.

3.16 Jewelry Planner Design

The Jewelry Planner is designed to accept budget, occasion, and style preferences. It also provides optional image input so the AI can consider an outfit's colors and aesthetics. This module can therefore use both text and image information.

3.17 Jewelry Recommendation Route

The /generate-jewelry route is designed for style-matched jewelry suggestions. The output is intended to reflect the user's occasion, preferred style, outfit context, and budget.

3.18 Shared Design Principle

All three planners follow a common design pattern: collect user information, organize the input, generate an appropriate AI prompt, process the recommendation request, and display structured results. This shared pattern supports consistency across the application.

Page 4 - Authentication, Session, and Interface Design

3.19 Registration and Login Design

The application design includes separate registration and login functions. The /register route handles new user registration, while /login authenticates existing users. /logout terminates the current user session and redirects the user to the login page.

3.20 Token and Session Design

The /token route is designed to issue a secure JWT token after successful authentication so that protected API routes can be accessed by authorized users. /session-info provides session metadata such as login status, while /session-data provides session-specific information used for personalization and recommendation tracking.

3.21 Dashboard Design

The User Dashboard is designed for logged-in users. It displays recent recommendations, saved queries, and personalized insights, creating a central place for reviewing previous activity.

3.22 Recommendation History Design

The Recommendation History page is designed to maintain a log of the user's past recommendation queries and results. The /history route supports retrieval of this information so previous plans can be reviewed or reused.

3.23 Recommendation Page Design

Each planner has a corresponding recommendation display page. The design uses structured and card-like layouts so recommendations can be shown with product suggestions, item details, prices, and relevant information.

3.24 Responsive UI Design

The frontend is designed to be responsive and user-friendly. The documentation emphasizes a lightweight interface, dynamic result display, clean forms, and support for real-time interaction between the frontend and backend.

3.25 CORS and Routing Design

CORS configuration is included in the backend design to support communication between frontend and backend where cross-origin requests are required. Static routing and startup initialization are also included in the design.

Page 5 - AI Workflow, Data Flow, and Final Design

3.26 AI Prompt Design

The project design uses planner-specific prompts and templates. The AI workflow is intended to format the user's budget, identify the selected domain, interpret preferences, and generate recommendations appropriate to that domain. The documentation also calls for prompt refinement to improve recommendation quality.

3.27 Multimodal Design

The Jewelry Planner supports optional image input in addition to text-based information. The design allows Gemini to consider an outfit image together with the stated occasion, style preferences, and budget when producing jewelry suggestions.

3.28 Recommendation Detail Design

The /recommendations-details route is designed to return detailed AI-generated recommendations based on the user's budget, preferences, and selected category. This supports a more detailed result view after the main recommendation request.

3.29 Startup and Application Entry Design

The /startup function initializes essential application services and configuration when the FastAPI server starts. The application also includes a main entry point for starting the server using Uvicorn when the script is run directly.

3.30 Error Handling and Fallback Design

The project workflow includes validation and fallback behavior. Inputs should be checked to reduce failures caused by edge cases, and fallback or default recommendations can be used when the AI does not return sufficient results.

3.31 Testing-Oriented Design

The design supports testing across all three planners using different budgets and preferences. The documented testing scope includes home decor scenarios, party packages involving food, venue, and decoration, and jewelry recommendations using outfit styles, occasions, and image inputs.

3.32 Final Design Summary

The final Project Design Phase combines the major requirements into one modular system architecture. The frontend collects inputs, FastAPI routes process requests, planner modules organize domain-specific logic, Gemini AI generates personalized recommendations, and authentication and session features support individual user interaction. The design also includes recommendation history, detailed results, responsive pages, and testing support.

3.33 Conclusion

The Project Design Phase provides the technical and structural blueprint for implementing PocketSmart AI. It defines the relationship between the interface, backend, AI layer, planner modules, authentication functions, and recommendation flow. This design prepares the project for implementation while keeping the system organized and focused on budget-aware personalization.